Esercizi — selezione

Lezione coperta: `L05` — selezione: `if`, `if ... else`, `if ... elif ... else`, condizioni annidate, `pass`

Variabili, operatori ed espressioni hanno una scheda loro, `L04_esercizi.md`. Qui si dà per saputo tutto quello: il nuovo è **scegliere**.

Come si lavora. Tre gradini: prima **seguire** un codice e dire quale ramo scatta, poi **trovare** l'errore, infine **scriverne** uno. Le soluzioni sono sotto ogni esercizio: aprile dopo aver provato.

Il primo gradino si fa senza calcolatore. Prevedere quale ramo verrà eseguito è l'unica cosa che questa lezione chieda davvero di imparare.

Gradino 1 — seguire il codice

Esercizio 1 — Il maggiore dei due (difficoltà: •)

Che cosa stampa, per ciascuna delle tre coppie?

```
a = 7
b = 7

if a > b:
    print("il maggiore è a")
else:
    print("il maggiore è b")
```

Coppie: `a=7 b=7`, poi `a=9 b=2`, poi `a=2 b=9`.

Soluzione

Mostra la soluzione

a	b	stampa
7	7	il maggiore è b
9	2	il maggiore è a
2	9	il maggiore è b

Il caso interessante è il primo. Quando i due valori sono uguali il programma dice che il maggiore è `b`, che è falso: non c'è un maggiore.

Non è un errore di Python, è un buco nel ragionamento. `else` non significa «`b` è più grande», significa «tutto ciò che non è `a > b`» — e lì dentro ci sta anche il caso in cui sono uguali. Per coprirlo servono tre rami, non due.

Esercizio 2 — Quale ramo scatta (difficoltà: •)

```
if t > 30:
    print("molto caldo")
elif t > 20:
    print("caldo")
elif t > 10:
    print("mite")
else:
    print("freddo")
```

Che cosa stampa per `t` uguale a 35, 25, 15, 5 — e poi per **30** e **20**?

Soluzione

Mostra la soluzione

`t`	stampa
35	molto caldo
25	caldo
15	mite
5	freddo
30	**caldo**
20	**mite**

I primi quattro sono quelli che ti aspetti. **Gli ultimi due sono il punto dell'esercizio**: 30 non è «molto caldo» perché `30 > 30` è falso, e 20 non è «caldo» per la stessa ragione.

I valori esatti ai confini sono dove si annidano gli errori, e non si vedono provando a caso: bisogna provarli apposta. Se volevi includerli servirebbe `>=`.

Esercizio 3 — L'indentazione decide (difficoltà: •)

Due programmi che differiscono per **quattro spazi**. Che cosa stampa ciascuno per `voto = 10`?

```
# versione A
if voto >= 18:
    print("promosso")
print("fine")
```

```python
# versione B
if voto >= 18:
    print("promosso")
    print("fine")
```

Soluzione

Mostra la soluzione

Con `voto = 10`:

- **versione A** stampa `fine`
- **versione B** non stampa **niente**

In A la riga `print("fine")` è fuori dal blocco, quindi si esegue sempre. In B è dentro, quindi si esegue solo quando la condizione è vera.

In Python l'indentazione **non è formattazione**: è la sintassi che dice dove comincia e dove finisce un blocco. In altri linguaggi quel compito ce l'hanno le parentesi graffe, e l'indentazione è solo cortesia verso chi legge. Qui no.

Nota che con `voto = 30` le due versioni stampano la stessa cosa: l'errore si manifesta **solo su un ramo**, ed è il motivo per cui va provato anche il caso che non ti interessa.

Esercizio 4 — `elif` o due `if` (difficoltà: •)

Che cosa stampa ciascuno dei due, con `n = 15`?

```python
# versione A
if n % 3 == 0:
    print("multiplo di tre")
if n % 5 == 0:
    print("multiplo di cinque")
```

```python
# versione B
if n % 3 == 0:
    print("multiplo di tre")
elif n % 5 == 0:
    print("multiplo di cinque")
```

Soluzione

Mostra la soluzione

- **A** stampa **due righe**: `multiplo di tre` e `multiplo di cinque`
- **B** stampa **una riga sola**: `multiplo di tre`

`elif` vuol dire «**altrimenti**, se»: viene esaminato solo quando le condizioni precedenti sono risultate false. Due `if` separati sono invece due domande indipendenti, e possono essere vere entrambe.

La scelta non è stilistica, dipende da che cosa stai chiedendo. «In quale fascia cade questo valore?» ha una risposta sola, e vuole `elif`. «Quali di queste proprietà ha?» può averne più d'una, e vuole `if` separati.

Gradino 2 — trovare l'errore

Esercizio 5 — Un simbolo di troppo (difficoltà: ••)

Questo programma non parte. Perché?

```
voto = 18
if voto = 18:
    print("esattamente diciotto")
```

Soluzione

Mostra la soluzione

```
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

= **assegna**, == **chiede**. Dentro un `if` serve una domanda, non un'istruzione.

È l'errore più comune di chi arriva dalla matematica, dove il simbolo è uno solo. Python in questo caso aiuta: il messaggio dice esattamente che cosa hai sbagliato, e vale la pena leggerlo invece di fissare il codice.

Esercizio 6 — Un ramo che non scatta mai (difficoltà: ••)

Questo programma non dà errori, ma è sbagliato. Provalo con `t = 35` e con `t = 25`, poi spiega.

```
if t > 10:
    print("mite")
elif t > 20:
    print("caldo")
elif t > 30:
    print("molto caldo")
else:
    print("freddo")
```

Soluzione

Mostra la soluzione

Stampa **`mite`** per 35, per 25 e per 15. I rami `caldo` e `molto caldo` **non si eseguono mai**, qualunque sia `t`.

La catena si ferma al primo ramo vero, e `t > 10` è vera per ogni valore che soddisferebbe anche gli altri due. Quelle due righe sono codice morto: ci sono, si leggono, e non servono a niente.

Nessun errore, nessun avviso. Il programma gira e risponde, e la risposta è sbagliata per tutti i valori sopra 10.

La regola: in una catena di `elif` le condizioni vanno dalla **più stretta alla più larga**. Se le scrivi al contrario, la più larga mangia tutte le altre.

Esercizio 7 — Lo script del 2009 (difficoltà: ••)

Questo programma viene da una vecchia raccolta di esercizi. Doveva chiedere due numeri e dire se la loro somma supera 100. Eseguilo: non funziona. Trova i **due** problemi e correggilo.

```
a = input("inserisci un numero da 0 a 100 ")
b = input("inserisci un numero da 0 a 100 ")
somma = a + b
if somma > 100:
    print("Numero troppo alto")
else:
    print("Numero corretto")
```

Soluzione

Mostra la soluzione

Battendo `40` e `50`:

```
TypeError: '>' not supported between instances of 'str' and 'int'
```

Primo problema: `input()` restituisce sempre stringhe, quindi `a + b` non somma, concatena: `somma` vale `'4050'`.

Secondo problema: l'errore arriva una riga dopo, quando si prova a confrontare quella stringa con `100`.

I due problemi sono in realtà **lo stesso**, visto due volte, e si correggono in un punto solo — convertendo appena si legge:

```
a = int(input("inserisci un numero da 0 a 100 "))
b = int(input("inserisci un numero da 0 a 100 "))
somma = a + b
if somma > 100:
    print("Numero troppo alto")
else:
    print("Numero corretto")
```

Da notare: con `40` e `50` il programma sbagliato si **ferma** con un errore. Se il confronto fosse stato fra due stringhe non si sarebbe fermato affatto, e avrebbe risposto confrontando parole invece di numeri.

Esercizio 8 — Il blocco vuoto (difficoltà: ••)

Perché questo programma non parte, e qual è il modo giusto di scriverlo?

```
if x >= 0:
else:
    print("negativo")
```

Soluzione

Mostra la soluzione

Python si aspetta un blocco indentato dopo i due punti, e non lo trova: `IndentationError: expected an indented block`.

Un blocco non può essere vuoto. Se davvero non deve succedere niente esiste `pass`, che è un segnaposto:

```
if x >= 0:
    pass
else:
    print("negativo")
```

Ma qui non è la soluzione giusta, è solo quella che fa partire il programma. Un `if` con il ramo vero vuoto è una condizione scritta al contrario: si gira.

```
if x < 0:
    print("negativo")
```

`pass` serve mentre stai scrivendo, quando il codice di un ramo non c'è ancora e vuoi provare il resto. Lasciarlo nella versione finale è un segnale che qualcosa non è stato finito o non è stato ripensato.

Gradino 3 — scrivere

Esercizio 9 — Il maggiore, per davvero (difficoltà: •••)

Riprendi l'esercizio 1 e scrivilo in modo che risponda correttamente anche quando i due numeri sono uguali.

Soluzione

Mostra la soluzione

```
if a > b:
    print("il maggiore è a")
elif b > a:
    print("il maggiore è b")
else:
    print("sono uguali")
```

Tre casi, tre rami. L'`else` finale copre esattamente quello che resta: non «sono uguali» perché l'hai scritto tu, ma perché è l'unica possibilità rimasta dopo aver escluso le altre due.

Si poteva anche scrivere `elif a == b` al posto dell'`else`, ma allora servirebbe un quarto ramo per un caso che non esiste — e chi legge si chiederebbe quale sia.

Esercizio 10 — La fascia di temperatura (difficoltà: •••)

Scrivi un programma che legge una temperatura e stampa la fascia:

fascia	quando
molto caldo	30 gradi o più
caldo	da 20 a 29
mite	da 10 a 19
freddo	sotto 10

Attenzione ai valori esatti: 30 deve dare «molto caldo», 20 «caldo».

Soluzione

Mostra la soluzione

```
t = int(input("temperatura in gradi: "))

if t >= 30:
    print("molto caldo")
elif t >= 20:
    print("caldo")
elif t >= 10:
    print("mite")
else:
    print("freddo")
```

Rispetto all'esercizio 2 cambia una cosa sola: `>=` invece di `>`. È quello che sposta 30 e 20 nella fascia giusta.

Due cose che rendono il programma più semplice di quanto sembri. Le condizioni sono dalla **più stretta alla più larga**, quindi la catena funziona; e nessun ramo deve dire «fra 20 e 29», perché arrivarci significa già che `t` è minore di 30 — lo ha stabilito il ramo precedente.

Esercizio 11 — Tre lati (difficoltà: •••)

Scrivi un programma che legge tre numeri e dice se possono essere i lati di un triangolo. La regola: **ciascun lato deve essere minore della somma degli altri due**.

Verificalo su `3 4 5`, `1 2 10`, `2 2 4` e `5 5 5`.

Soluzione

Mostra la soluzione

```
a = int(input("primo lato: "))
b = int(input("secondo lato: "))
c = int(input("terzo lato: "))

if a + b > c and a + c > b and b + c > a:
    print("possono essere i lati di un triangolo")
else:
    print("non possono")
```

lati		perché
3 4 5	sì	ogni somma supera il terzo
1 2 10	no	1 + 2 non arriva a 10
2 2 4	**no**	2 + 2 fa esattamente 4, e serve **maggiore**
5 5 5	sì	equilatero

Il caso `2 2 4` è quello che distingue chi ha capito: la somma è uguale, non minore, e un «triangolo» con quei lati sarebbe un segmento. Se avessi scritto `>=` l'avresti accettato.

Servono tutte e tre le condizioni: controllarne una sola non basta.

Esercizio 12 — Da annidato a concatenato (difficoltà: •••)

Questo programma funziona. Riscrivilo usando `elif`, senza annidare, e verifica su tre coppie che le due versioni rispondano allo stesso modo.

```
if x == y:
    print("sono uguali")
else:
    if x < y:
        print("x è minore")
    else:
        print("x è maggiore")
```

Soluzione

Mostra la soluzione

```
if x == y:
    print("sono uguali")
elif x < y:
    print("x è minore")
else:
    print("x è maggiore")
```

Verificato su `(3,3)`, `(2,5)` e `(7,1)`: le due versioni danno la stessa risposta in tutti e tre i casi, e la darebbero per qualunque coppia.

`elif` **è** esattamente questo: un `else` che contiene un solo `if`, scritto in modo che non si allontani dal bordo. Con tre rami la differenza è piccola; con sei, la versione annidata arriva a sei livelli di indentazione e diventa illeggibile.

Il contrario non vale sempre: se dentro l'`else` ci fosse **altro** oltre all'`if` — un'istruzione prima o dopo — la riscrittura con `elif` non sarebbe possibile.